TASK 3: Data Types and Type Compatibility in C++

Name: Nagineni Sunil Kumar
Register Number: 192472330
Subject: Data Structures and Algorithms (DSA)

1. Aim

To study C++ data types, memory sizes, numeric ranges, type compatibility, and the use of suitable data types for the Metro Smart-Card Fare System.

2. Theory

Different data types are used to store different kinds of values in C++.

Data Type	Purpose	Example
int	Stores integers	int hour = 8;
long long	Stores large integers	long long balance = 10000;
float	Stores approximate decimal values	float rate = 2.5f;
double	Stores decimal values with greater precision than float	double fare = 10.50;
char	Stores a character	char zone = 'A';
bool	Stores true or false	bool peak = true;
auto	Allows the compiler to infer a variable's type	auto count = 10;

3. Data Types for the Metro System

Requirement	Recommended Type	Reason
Card balance	long long	Stores balance in paise
Fare amount	long long	Stores fare in integer paise
Journey hour	int	Stores hours from 0 to 23
Peak-hour status	bool	Stores true or false
Metro zone	enum class Zone	Represents the available zones
Journey count	long long	Supports large journey totals

Money can be stored in integer paise to avoid many floating-point rounding issues. For example, ₹10.50 is represented as 1050 paise.

5. Expected Output

On a typical system where int is 4 bytes and double and long long are 8 bytes, the output is:

Size of int: 4 bytes

Size of double: 8 bytes

Size of long long: 8 bytes

INT_MIN: -2147483648

INT_MAX: 2147483647

Journeys: 3000000

Fare in paise: 1000

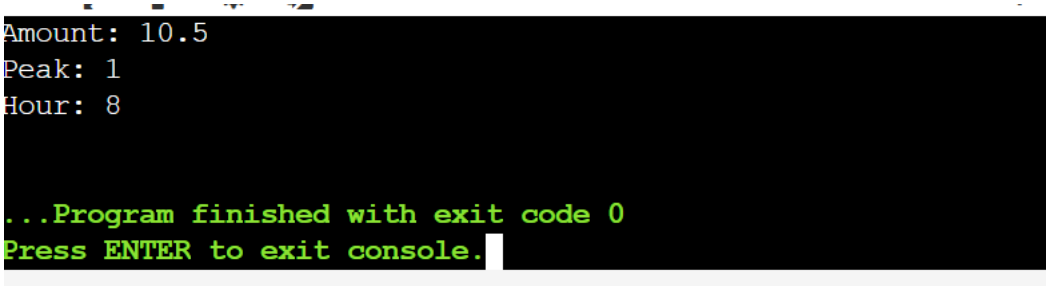
Amount: 10.5

Peak: 1

Hour: 8

Note: Check the actual output on your compiler because data type sizes can vary by implementation.

Obtained output:



6. Type Compatibility

Type compatibility determines how values of different data types behave in expressions.

Example:

int a = 7;

int b = 2;

cout << a / b << '\n';

cout << static_cast<double>(a) / b << '\n';

Output:

3

3.5

The first expression uses integer division. The second converts a to double, allowing decimal division.

7. Large Network Total and Overflow

Suppose 3,000,000 journeys each cost ₹30.

Total fare:

3,000,000 × ₹30 = ₹90,000,000, or ₹9 crore.

In paise:

3,000,000 × 3000 = 9,000,000,000 paise.

This value exceeds the maximum of a typical 32-bit signed int, which is 2,147,483,647. Therefore, long long is a more suitable choice for storing this total.

8. Time and Space Complexity

- Time Complexity: O(1), because the program performs a fixed number of operations.
- Space Complexity: O(1), because it uses a fixed number of variables.

For a program that processes (n) journeys, calculating the total takes O(n) time.

9. Conclusion

C++ data types must be selected according to the values being stored. Integer paise and long long are suitable for fare calculations and large network totals. Using sizeof, <climits>, and explicit type conversions helps programmers understand memory usage, numeric ranges, and type compatibility.